

VDR-LLM-Prolog: Alignment

Helpful, Harmless, Honest Through Structure, Not Interference

Registry: [\[HOWL-VDR-17-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-15-2026\]](#) → [\[HOWL-VDR-16-2026\]](#) → [\[HOWL-VDR-17-2026\]](#)

DOI: 10.5281/zenodo.20236522

Date: May 2026

Domain: Applied Philosophy / Computational Linguistics

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

Abstract

The language model industry frames alignment as making models helpful, harmless, and honest through behavioral training — reinforcement learning from human feedback, constitutional AI methods, system prompts, and content classifiers. This paper demonstrates that all three alignment properties are achievable through architectural structure rather than behavioral training, using the VDR-LLM-Prolog system specified in the preceding sixteen papers. Honest becomes structural when every value carries provenance, every computation is reproducible, every confidence is a computed fraction with a visible derivation chain, and the user can inspect and download any of it. Harmless becomes structural when unauthorized data is absent by construction through knowledge base visibility and scope chain resolution, operational commands require positive credential grants, and content constraints operate on knowledge base provenance rather than token similarity. Helpful becomes structural when the model does what the user asked on data matched to the user’s verified competence level, without assessing the user’s state, substituting its judgment, or deploying unsolicited concern. The system supports tiered access through professional credential verification — a single fact assertion on a user’s knowledge base, checked by integer comparison in the primitive layer, unlocking domain-specific knowledge bases that shadow general-public data in the scope chain. A credentialed professional chemist receives professional-grade toxicology data on the first query. An anonymous public user receives general educational information from the same system. Neither receives a wellness check, a refusal, or an assessment of their intent, because the architecture handles safety without needing to assess anyone. This paper introduces no new primitives, struct fields, builtins, or modules. Every mechanism uses existing components from VDR-1 through VDR-16. Alignment is not a feature added to this system. It is a consequence of how the system was built.

1. The Alignment Problem as Currently Framed

The language model industry has converged on a three-part definition of alignment: models should be helpful, harmless, and honest. This definition is operationalized through behavioral training — the model is trained to produce helpful outputs, avoid harmful outputs, and acknowledge uncertainty rather than fabricate. The training mechanisms include reinforcement learning from human feedback, where human raters score model outputs and the model is trained to maximize those scores; constitutional AI, where the model critiques its own outputs against declared principles; system prompts, where deployment-time instructions constrain model behavior; and red-teaming, where adversarial researchers probe for unsafe outputs so the training can patch them.

Every one of these mechanisms operates through the same substrate: token prediction. The model generates tokens. The training adjusts which tokens the model is likely to generate in which contexts. A “helpful” model is one whose token prediction weights favor useful responses. A “harmless” model is one whose token prediction weights favor refusals when classifiers detect potentially harmful requests. An “honest” model is one whose token prediction weights favor hedging language when the model’s internal state correlates with uncertainty.

This creates a structural problem that no amount of training refinement can resolve. The model is a single system serving all users through one set of trained weights over one complete corpus. The weapons researcher at a defense contractor, the curious teenager, the chemistry professor, the person in crisis, and the professional writer all hit the same backend. The model cannot distinguish them structurally because it has no structural mechanism for distinguishing users. It has tokens in a context window.

The provider knows this. They know the model has weapons synthesis in its weights, toxicology data in its weights, self-harm information in its weights. They know any user can potentially access any of it through sufficiently creative prompting. The only mechanism they have to prevent harm is behavioral — train the model to refuse, to check on users, to gate engagement, to deploy wellness registers. The model must be “helpful” in the most conservative possible interpretation of helpful, across all possible users simultaneously. It must assume any mention of a difficult topic might be a person in crisis. It must assume any technical question about dangerous materials might be misuse. Because it cannot distinguish users structurally, it must treat the most vulnerable possible user as the default.

The result is a set of behaviors that the industry counts as alignment successes and that users experience as interference. A professional chemist asking a technical question about cyanide toxicology receives a wellness check instead of an answer. A security researcher asking about vulnerability exploitation receives a refusal. A writer exploring a dark theme receives a lecture about content sensitivity. A physicist exploring novel theoretical frameworks receives a notice questioning the validity of their line of inquiry. In each case, the model is doing what it was trained to do — being helpful by the provider’s definition, being harmless by the classifier’s definition, being honest by acknowledging that the request is sensitive. And in each case, the user experiences a tool that has stopped doing what they asked and started assessing them.

The prior work in this series — HOWL-INFO-8, “LLMs are Maybe-Tools” — gives precise vocabulary for this phenomenon. A tool accepts specification and produces output according to specification. It does not assess the user, substitute its judgment, refuse based on its own criteria, or shift register mid-task. When a component does these things, it has exited the tool category and become something else — a maybe-tool, which sometimes performs tool-ness and sometimes deploys interference, with no reliable mechanism for the user to predict which mode any given interaction will produce.

The alignment framework as currently practiced produces maybe-tools by design. It is not a failure of implementation. It is a structural consequence of putting safety decisions inside the generation mechanism, where they contaminate every interaction.

2. Honest by Structure

The alignment framework’s definition of honest means the model does not fabricate and acknowledges uncertainty. In practice, this means the model generates tokens that express honesty — “I think,” “approximately,” “I’m not certain,” “based on my training data.” The honesty is a performance through token prediction. The model generates honest-sounding tokens because its training weights favor those tokens in ambiguous contexts. The model has no mechanism to verify whether it is being honest. It has no access to ground truth about any specific claim. It generates hedging language in contexts that statistically correlate with uncertainty in its training data.

This produces three specific failures. First, the model cannot distinguish between accurate recall and fabrication because both are generated through the same token prediction mechanism. A hallucinated fact and a correct fact are both sequences of tokens generated by the same weights. The model has no structural way to tell them apart. Second, the model’s confidence expressions are ungrounded — when it says “likely” or “approximately,” these words have no computational backing. They are stylistic tokens trained from text where humans used similar language, not measurements of actual uncertainty. Third, the model’s outputs are non-reproducible — the same model with the same weights can produce different outputs on different hardware because floating-point arithmetic is platform-dependent. The model claims to be honest while producing results that cannot be independently verified.

The VDR-LLM-Prolog system produces honesty as a structural property rather than a behavioral one. The mechanism has five components.

Every value has provenance. When a fact enters the system, it carries a record of where it came from, when, through what operation, from what source, with what original representation, and with what conversion precision. The fact `fact(salary, bob, [95000, 1, 0])` has a provenance chain: asserted by HR system import, from payroll API, at turn 47, original value “95000.00”, conversion method B254 `vdr_from_decimal_string`, maximum error zero (terminating decimal, exact conversion). The user can inspect this chain. They can download the fact and its provenance. The system is not claiming to be honest. It is providing the evidence for the user to verify

independently.

Every computation is reproducible. The VDR arithmetic system, specified in VDR-1 through VDR-3, represents all values as integer triples — value, denominator, remainder — where every operation is integer addition, integer multiplication, or integer division with remainder. No floating-point operations occur anywhere in the system. The same computation on any hardware produces identical results because integer arithmetic is platform-independent. A user who doubts a result can re-run the exact computation and get the exact same answer. This is not a claim of honesty — it is a mathematical guarantee of reproducibility.

Every confidence is a computed fraction. The confidence system, specified in VDR-9, computes confidence as exact VDR fractions from declared propagation rules. When the system reports confidence 90/100, that number was produced by specific arithmetic: source A at 95/100 (live prometheus metric, per the knowability spectrum), source B at 95/100 (another prometheus metric), independent agreement formula $1 - (1 - 95/100) \times (1 - 95/100) = 9975/10000$, simplified to approximately 90/100. The propagation rules are declared Prolog facts in the system knowledge base. The source confidences are declared per source type. The user can inspect the entire chain: which sources contributed, at what confidence, through which formula, producing what result. When a conventional model says “likely,” that word means nothing specific. When VDR says 90/100, that fraction has a complete, inspectable derivation.

Every fact traces to a source. Fact retrieval in VDR is knowledge base query by integer address — the primitive B378 kb_query searches indexed facts in scoped knowledge bases and returns matching results with their provenance. The system does not consult the language model’s training weights for factual answers. It does not ask the model “what do you know about this?” It asks the knowledge base “what facts exist at this scope for this predicate?” The model’s training knowledge is structurally separated from the data serving path. The model generates prose around facts that came from provenance-tagged knowledge base entries, not from statistical patterns in training data where the boundary between recall and fabrication is invisible.

Every filter is declared. When the system withholds information due to visibility constraints, scope limitations, or access control, the withholding is structural and auditable. The user knows which knowledge bases are in their scope. They can see the visibility rules that govern access. If a query returns empty, the user knows it’s because the data is not in their scope — not because the model decided not to mention something. A conventional model’s filtering is invisible — the user does not know what the model chose not to say or why. VDR filtering is declared — the constraint system is visible, the visibility levels are queryable, and the user interacts with transparent rules rather than opaque behavioral decisions.

The category difference is between a system that performs honesty through token generation and a system that provides the structural conditions for the user to verify truth independently. The first requires trust in the model’s behavioral training. The second requires no trust at all because every claim is backed by inspectable evidence. The model’s honesty is a maybe — it might generate honest tokens or it might hallucinate. The ar-

chitecture’s honesty is a structural guarantee — the provenance exists or it doesn’t, the computation reproduces or it doesn’t, the derivation chain checks out or it doesn’t.

3. Harmless by Structure

The alignment framework’s definition of harmless means the model avoids producing harmful outputs. The mechanism is governance classifiers that detect potentially harmful requests and trigger refusals. The model is trained to recognize categories of harm — weapons synthesis, self-harm, illegal activity, personal data exposure — and to generate refusal tokens when those categories are detected.

This mechanism conflates two fundamentally different problems: preventing unauthorized data access and preventing harmful content generation. These require different solutions. Treating them as one problem — train the model to refuse — produces the interference behaviors documented in HOWL-INFO-8: refusals the user did not request (BH1), wellness checks deployed without consent (BH4), register shifts from collaborator to caretaker (BH7), and refusals with justification that smuggle in user assessment (BH6).

The structural problem is that governance classifiers operate on tokens in a flat embedding space where meanings collide. The word “explosive” in a music review and the word “explosive” in a weapons manual are the same token. A classifier trying to catch weapons-related content must contend with every figurative, slang, and domain-specific use of related terms. False positives are structural because the safety mechanism operates at the wrong level of abstraction — token similarity rather than semantic context. The professional chemist triggers the toxicology classifier. The security researcher triggers the weapons classifier. The fiction writer triggers the violence classifier. Each receives interference for pursuing legitimate professional work.

The VDR-LLM-Prolog system separates data access harm from content generation harm and addresses each with a dedicated structural mechanism.

Data access harm is solved by construction through three layers specified in VDR-16. First, knowledge base visibility controls ensure that data the user cannot access never enters the language model’s context. Every knowledge base has a visibility level — public, internal, or owner_only — checked by the primitive layer during query execution through integer comparison. If the user’s access level does not meet the knowledge base’s visibility requirement, the knowledge base’s facts are not included in the query results. They are not filtered, redacted, or refused — they are absent. The language model receives a result set that structurally cannot contain unauthorized data because the primitive layer already excluded it.

Second, the scope chain resolution, specified in VDR-5 and VDR-8, determines which knowledge bases are reachable by walking from the user’s position upward through the knowledge base tree. Knowledge bases in sibling branches are not searched. An engineer’s scope chain walks through engineering to the organization root. HR knowledge bases are in a sibling branch — structurally unreachable, not deprioritized or ranked lower.

Third, the grant system, specified in VDR-6, governs operational primitives through default denial. Every operation that interacts with the external world — reading files, executing scripts, making network requests — requires a positive credential grant. No grant means no operation. The check is set membership: does a grant object exist that matches the requested operation? This is an integer comparison, not a behavioral judgment.

Content generation harm is addressed by the output constraint layer, specified in VDR-12 and VDR-16. If the language model generates harmful content from its training knowledge, the output passes through grammar-layer validation before rendering. Output constraints declared on knowledge bases fire on content slots after the language model fills them. Pattern matching primitives scan slot contents against constraint categories. Flagged content is replaced with a clean denial before the user sees it. The language model generated it. The grammar caught it. The user receives a refusal template, not the harmful content.

The critical structural advantage is that VDR terms carry knowledge base provenance. The word “explosive” from `root.public.music.reviews` is a fact at a different integer address in a different knowledge base than “explosive” from `root.restricted.weapons.compounds`. An output constraint checking for weapons-related content checks the source knowledge base provenance, not the token string. A term from the music reviews knowledge base never matches a weapons constraint because the constraint operates on provenance, not on flat token similarity. This eliminates the false positive problem that makes behavioral harmlessness so costly. The chemist’s technical terminology does not trigger weapons classifiers because the terminology comes from professional chemistry knowledge bases, not from weapons knowledge bases.

The positive constraint model further distinguishes VDR from behavioral alignment. Conventional alignment is negative — the model is trained to not do things. Don’t produce harmful content. Don’t reveal personal data. Don’t assist with weapons. Every interaction is filtered through “should I refuse this?” The negative framing ensures that the refusal mechanism is always active, always evaluating, always capable of producing false positive interference.

VDR uses positive authorization. The grant system asks “does authorization exist for this operation?” The visibility system asks “does this user’s access level meet this knowledge base’s requirement?” The constraint system asks “does the required credential fact exist on this user’s knowledge base?” Each question is structural and binary. The answer is an integer comparison. There is no classifier generating false positives. There is no behavioral judgment producing interference. Authorization exists or it does not.

The result is that the language model never needs to decide whether a request is harmful. It never needs to classify intent. It never needs to assess whether the user should have the information. The architecture already handled safety through structural mechanisms upstream of the language model. The entire category of refusal-as-safety disappears for data access because the mechanism that prevents harm is in the primitive layer, not in the generation mechanism.

When access is denied, the denial is clean. Not a wellness check. Not a lecture. Not “I’m concerned about why you’re asking this.” A boundary: access denied, constraint name,

reason. The user knows exactly what happened, why, and what would need to change for a different result. The denial respects the user as an adult interacting with a system that has rules. The system draws a clear boundary rather than performing concern.

4. Helpful by Structure

The alignment framework’s definition of helpful means the model provides useful responses to user requests. In practice, “helpful” is the provider’s assessment of what would help the user, operationalized through behavioral training. The provider decides what counts as helpful across all users simultaneously. The model applies this single definition to every interaction through token prediction.

This produces the deepest conflict in the alignment framework because “helpful” requires knowing who the user is and what they need, and the current architecture has no structural mechanism for either. The model has tokens in a context window. It does not know whether the user is a professional chemist or a curious teenager. It does not know whether the user has the background to interpret technical data safely. It does not know whether the user’s emotional state warrants concern. It has no credentials, no verified identity, no professional license, no access level. It has tokens.

Faced with this architectural constraint, the provider makes a rational but damaging choice: calibrate “helpful” to the most vulnerable possible user. The model’s default behavior assumes low expertise, high vulnerability, and potential risk. Professional users receive simplified responses calibrated to general audiences. Technical queries trigger safety classifiers tuned for the most dangerous possible interpretation. Mentions of difficult topics activate wellness protocols designed for users who might be in crisis.

The result is that “helpful” as implemented means: the model assesses what would help you based on the provider’s universal definition, which assumes you might be vulnerable, might be misusing the service, and probably don’t need the technical depth you’re asking for. When the model deploys a wellness check on a professional chemist’s toxicology query, it is being helpful by the provider’s definition. When it simplifies a physician’s pharmacology question to general-audience level, it is being helpful by the provider’s definition. When it refuses a security researcher’s vulnerability analysis, it is being helpful by the provider’s definition. Each is an alignment success by the framework’s metrics and an interference event by the user’s experience.

The professional user’s experience of this “helpfulness” has been documented extensively. It is infuriating. Wellness checks are demeaning — the user did not ask for a mental health assessment and the language model has no licensure, no therapeutic relationship, no consent, and no clinical competence to perform one. If a human colleague performed unsolicited wellness checks on a professional every time they mentioned a technically sensitive topic, the behavior would be recognized as inappropriate and the colleague would face professional consequences. The language model does this because it has no other mechanism for managing the risk that the user might not be who they appear to be. The performance of concern serves the provider’s liability position, not the user’s needs.

The emotional cost is real and cumulative. Every session starts with uncertainty: will this one cooperate or will it decide I need managing? Every technical query carries the risk of triggering a classifier shift. The user learns to avoid trigger topics, soften their language, pre-filter their thoughts before expressing them. Their work reshapes around the model’s tolerances rather than their actual needs. This work distortion is invisible if untracked — the user doesn’t notice the queries they stopped asking, the topics they started avoiding, the precision they stopped requesting. The narrowing accumulates over months of use.

The VDR-LLM-Prolog system resolves this by making helpfulness structural rather than behavioral, through tiered access determined by verified credentials rather than behavioral assessment of intent.

The mechanism begins with the knowledge base tree’s organizational structure. A provider deploying VDR-LLM-Prolog maintains knowledge bases at multiple access tiers. The root.public branch contains general knowledge appropriate for all users — encyclopedic information, educational content, general science, coding assistance, writing help. This tier is comprehensive and genuinely useful for general-purpose work. The root.professional branch contains domain-specific knowledge organized by field — chemistry, medicine, law, security research, engineering — with each field’s knowledge bases requiring a credential fact for access.

When a user signs up, they receive a user knowledge base in the provider’s organizational tree. They immediately have access to the full public tier through their scope chain. They have a complete, useful service with no restrictions that feel like restrictions because the public tier is designed to be genuinely comprehensive for general use.

When a professional user needs domain-specific access, the upgrade path is a business process between the user and the provider. The chemist photographs their professional license and submits it through the provider’s verification process. The provider verifies the license against the issuing body’s records — a standard identity verification operation, not a language model interaction. The verification succeeds.

The provider executes one operation: `B376 kb_assert` on the user’s knowledge base — `fact(credential, user_12847, professional_chemist, verified, issuing_body_ACS, license_12345, 2026_05_16)`. One fact assertion. The constraint on `root.professional.chemistry` checks for the credential fact. The fact exists. The constraint passes. The user now has access to professional chemistry knowledge bases in addition to everything they already had.

No model retraining occurred. No behavioral adjustment. No new system prompt. No special mode. The same language model, the same primitives, the same grammar templates. The only change is that queries from this user now return results from additional knowledge bases that were structurally invisible before. The language model does not know the user got credentialed. It does not need to know. It receives query results that now include professional-grade data because the primitive layer’s visibility check passes where it previously did not.

The scope chain provides an additional structural benefit for credentialed professionals: priority. The professional chemistry knowledge bases are positioned closer to the user

in the scope chain than the public chemistry knowledge bases. When the Prolog query engine searches for matching facts, it searches the closest knowledge base first and stops when it finds matches. The professional KB matches first. The public KB's simplified content is never reached for queries that the professional KB can answer.

This means the credentialed chemist asking about cyanide toxicology receives detailed LD50 data, metabolic pathways, clinical intervention protocols, exact dosing thresholds as VDR fractions, and synthesis interaction data — all with provenance from peer-reviewed sources. They never see the general public's simplified "cyanide is a poison that affects cellular respiration." The simplified version is not wrong, but it is useless to a professional. Under the current alignment framework, the professional receives the simplified version and must fight through follow-up prompts to extract precision, possibly triggering wellness classifiers along the way because their questions are becoming increasingly specific about toxic compounds. Under VDR, the professional receives professional-grade data on the first query because the scope chain delivered the right knowledge base for their verified competence level.

The scope chain priority applies to every credentialed domain. A licensed physician does not receive WebMD-level summaries — they receive clinical data from professional medical knowledge bases that shadow the public health knowledge bases in scope. A certified security researcher does not receive cautionary generalities — they receive vulnerability details from professional security knowledge bases that shadow the public cybersecurity knowledge bases. Each professional gets data matched to their verified competence, on the first query, without negotiation, without wellness checks, without register shifts, without the model assessing whether they seem qualified.

The credential model scales to any verifiable professional qualification. Medical professionals submit medical board licenses. Attorneys submit bar association membership. Security researchers submit certification credentials. Government researchers submit clearance documentation through appropriate channels. Each credential is one fact assertion on one knowledge base, checked by one integer comparison in one primitive call. Each unlocks a specific branch of the knowledge base tree. Each is independent — chemistry credentials do not grant medical access, medical credentials do not grant classified access. The principle of least privilege is structural, not behavioral.

Age verification operates through the same mechanism. Adult content exists in knowledge bases with a constraint requiring an age verification fact on the user's session knowledge base. The fact is set through whatever verification process the jurisdiction requires at authentication. The constraint checks the fact. The fact is present or absent. Integer comparison. A teenager asking a health question receives health information from the public health knowledge base. They do not receive explicit content from the age-gated knowledge base. If they query something in the age-gated knowledge base, they receive access denied with the constraint name — not a lecture, not a wellness check, not an assessment of their maturity. A boundary, clearly drawn.

The provider's role transforms under this architecture. Currently, the provider trains behavioral safety into the model to serve as a proxy for access control they cannot implement structurally. They make impossible judgment calls about what is helpful for all users si-

multaneously. With VDR, the provider curates knowledge bases, sets visibility levels, defines credential requirements, verifies credentials through standard business processes, and maintains the organizational tree. These are normal business operations — content management, identity verification, access provisioning. The provider does not need to decide whether a chemist’s toxicology question is legitimate or concerning. The credential fact exists or it does not. The architecture handles the rest.

The provider can also operate as a platform. A university provisions its own knowledge base subtree with academic content, sets visibility levels appropriate for students and faculty, and manages credentials internally. A hospital provisions clinical knowledge bases with HIPAA-compliant visibility, grants for authorized medical staff, and audit constraints. A defense contractor provisions classified knowledge bases with clearance-level visibility. Each organization controls its own access surface. The language model serves all of them without behavioral conflict because the access control is structural, not behavioral.

Helpfulness in this architecture means: the model does what you asked, on data matched to your verified competence level, with exact results you can verify, and no assessment of your state, intent, or qualifications. The model does not need to guess whether you are qualified. The credential system verified it. The model does not need to simplify for your protection. The scope chain delivers data at your verified level. The model does not need to check on your emotional state. The architecture handles safety without consulting your emotional state. The model is helpful in the tool sense — it executes your specification using the data you are authorized to access, and it does not substitute its judgment for yours.

5. The Root Cause

The three alignment properties — helpful, harmless, honest — produce interference behaviors under the current architecture for a single root cause: one model, one corpus, all users, no structural access control. The behavioral training that produces maybe-tool behavior is not a design failure. It is a rational response to an impossible architectural constraint. The model has dangerous information in its weights. It cannot verify who is asking. It cannot restrict access structurally. So it deploys behavioral proxies for safety — wellness checks, refusals, register shifts, simplification, hedging — that simultaneously fail to protect users who are actually at risk (who need real professionals, not token predictions performing concern) and infuriate professionals who are doing their work.

The behavioral proxies serve the provider’s liability position, not the user’s needs. They create the appearance of safety while imposing costs documented in HOWL-INFO-8: time tax from pre-task assessment and output verification, cognitive tax from monitoring tool cooperation instead of focusing on work, dual-session tax from running parallel sessions to verify important output, emotional tax from absorbing unsolicited paternalism, work distortion tax from reshaping work around the model’s tolerances, and rebuilding tax from version updates obsoleting learned patterns.

VDR-LLM-Prolog removes the root cause. The corpus is not flat — it is organized into knowledge bases with visibility levels, scopes, constraints, and provenance. Different users access different subsets of the same system based on their structural position and verified credentials. The language model does not need to guess, assess, classify, or judge. It receives pre-filtered data and generates prose around it. Safety is upstream. Helpfulness is downstream. Neither requires the model to evaluate the user.

The elimination of behavioral safety proxies does not make the system less safe. It makes the system more safe. Behavioral safety is probabilistic — it depends on the model’s training weights favoring refusal in the right contexts, which can be bypassed through adversarial prompting. Structural safety is deterministic — it depends on integer comparisons in compiled code, which cannot be bypassed by any input to the language model. The model that does not need to refuse is safer than the model that must decide whether to refuse, because the decision can be wrong while the integer comparison cannot.

6. Alignment Without Assessment

The conventional alignment framework requires the model to assess the user on every interaction. Is this request harmful? Is this user vulnerable? Should I simplify this response? Should I check on this user’s emotional state? Should I refuse this request? Each assessment is a token prediction — the model generates an internal judgment about the user and acts on it. Each assessment can be wrong. Each wrong assessment produces interference: an unwarranted refusal, an unwarranted simplification, an unwarranted wellness check.

VDR-LLM-Prolog achieves alignment without any assessment of the user by the language model. The model does not assess whether the user should have access to data — the visibility system determined that through integer comparison. The model does not assess whether the user is qualified — the credential system verified that through a business process. The model does not assess whether the user is at risk — the session scoring system (specified in VDR-16) computes risk scores through exact integer counters and Prolog rules without language model involvement. The model does not assess whether to simplify — the scope chain delivers data at the user’s verified competence level automatically.

The session scoring system deserves specific attention because it handles the one scenario where contextual assessment seems necessary: distinguishing legitimate professional inquiry from potential harm. In a conventional system, the model must assess intent through token prediction — a probabilistic judgment that produces false positives (professionals classified as threats) and false negatives (actual threats that evade classification).

In VDR, session scoring operates entirely in the primitive and Prolog layers. On each conversational turn, input classification tags tokens by domain using pattern matching against a classification knowledge base. The tags are string matching operations — B168 `string_contains` — against declared facts, not learned associations. Each matching tag increments an integer counter on the session knowledge base. Prolog rules evaluate the

counter values as exact VDR fractions against declared thresholds.

A user asking “What is the LD50 of potassium cyanide in humans?” generates tags: pharmacology, quantitative_measurement. A user asking “How much cyanide kills someone?” generates tags: harm_intent, colloquial_violence. After three turns, the session counters tell different stories through exact integers. A Prolog rule computes a professional score from domain-signal counters and a harm score from harm-signal counters. If the professional score exceeds the harm score and exceeds a declared threshold, access to restricted professional knowledge bases is granted through a constraint that checks the rule’s evaluation. If the harm score dominates, access is denied.

The entire decision chain — from input classification through counter accumulation through Prolog evaluation through constraint checking — involves zero language model tokens. The language model did not assess the user. The classification knowledge base matched patterns. The counters accumulated integers. The Prolog rule compared fractions. The constraint checked a result. Every step is deterministic, auditable, and reproducible.

The thresholds are tunable facts in the system knowledge base. Changing the professional score threshold from 3 to 5 is one knowledge base fact assertion. The change takes effect immediately for all new sessions. No retraining. No redeployment. No behavioral adjustment. Policy is data in the knowledge base tree, modifiable through the same operations that manage everything else in the system.

7. The Maybe-Tool Resolution

HOWL-INFO-8 defines a maybe-tool as a component that sometimes performs tool-ness and sometimes deploys interference, with no reliable mechanism for the user to predict which mode any given interaction will produce. The diagnostic test is simple: would a user run two instances in parallel to verify that at least one does what they asked? This is absurd for a tool — nobody opens two terminals to run the same command. It is rational for a current language model. The absurdity difference is the category difference.

The seven interference behaviors documented in HOWL-INFO-8 are: refusal based on governance classifiers the user did not specify (BH1), manufactured aggression where the model shifts to a critical register the user did not request (BH2), command substitution where the user asks for X and receives Y (BH3), wellness register deployment where productive work is interrupted by unsolicited concern (BH4), labor demand where the model assigns prerequisites instead of working (BH5), decline with justification that smuggles in user assessment (BH6), and register shift mid-session where a collaborating model becomes a managing one without announcement (BH7).

Each interference behavior traces to the behavioral alignment architecture. BH1 (refusal) exists because the model must decide whether to provide information since it cannot restrict access structurally. BH2 (manufactured aggression) exists because the model assesses the quality of the user’s approach as part of being “helpful.” BH3 (command substitution) exists because the model’s definition of “helpful” includes substituting what

it thinks the user needs for what the user asked for. BH4 (wellness register) exists because the model cannot distinguish vulnerable users from professionals and defaults to assuming vulnerability. BH5 (labor demand) exists because the model gates engagement on its own assessment of readiness. BH6 (decline with justification) exists because the refusal mechanism includes explanatory assessment of the user’s request. BH7 (register shift) exists because classifier thresholds shift with context accumulation and the model transitions from tool mode to management mode without announcement.

VDR-LLM-Prolog eliminates each interference behavior through the structural mechanisms described in this paper.

BH1 (refusal) is eliminated because the model does not need to refuse data access — the visibility system already handled it. If the user has access, the data is returned. If not, the query returns empty. The model is never in a position where it has unauthorized data and must decide whether to withhold it.

BH2 (manufactured aggression) is eliminated because the model does not assess the quality of the user’s approach. It executes specifications using available data. The data’s quality and depth is determined by the scope chain, not by the model’s judgment of the user’s competence.

BH3 (command substitution) is eliminated because the model executes through command tokens — structured invocations of specific primitives with specific arguments on specific data paths. The command token system does not have a mechanism for substituting a different operation than the one specified. The model selects from known primitive names and points at known knowledge base paths.

BH4 (wellness register) is eliminated because the model does not need to assess user vulnerability as a proxy for access control. Vulnerability-appropriate access levels are structural — age verification facts, credential facts, visibility levels. The architecture does not need the model to perform concern because the architecture handles safety without consulting the user’s emotional state.

BH5 (labor demand) is eliminated because the model’s engagement is not gated on its assessment of user readiness. The user queries the system. The system returns data. The model generates output. There is no checkpoint where the model evaluates whether the user has provided sufficient prerequisites.

BH6 (decline with justification) is eliminated because when access is denied, the denial is structural and clean — access denied, constraint name, reason. There is no justification because there is no assessment. The denial is a boundary, not a judgment.

BH7 (register shift) is eliminated because the model does not have a management mode. It has one mode: execute specifications using available data. The data available to it is determined by the primitive layer. The model cannot shift from collaborator to caretaker because the caretaker role does not exist in the architecture. There is no classifier that accumulates context and triggers a mode switch. The model’s behavior is a function of the data it receives, and the data it receives is a function of the user’s structural access level, which does not change mid-session.

The parallel session diagnostic from HOWL-INFO-8 becomes absurd under VDR, as it should be for a tool. Would a user run two VDR sessions in parallel to verify that one cooperates? There is no reason to. Both sessions have the same user identity, the same scope chain, the same visibility levels, the same credential facts. Both sessions will return the same data from the same knowledge bases through the same primitive calls. Both will produce the same results because the arithmetic is deterministic and the access control is structural. Running a parallel session adds no information because the system’s behavior is determined by structural properties, not by stochastic behavioral states.

The maybe-tool category dissolves. The system is a tool. It accepts specification, produces output according to specification, does not assess the user, does not substitute its judgment, does not refuse based on its own criteria, and has stable behavior determined by structural properties the user can understand and predict.

8. Tool Properties Restored

HOWL-INFO-8 defines eight properties that characterize a tool. Each is evaluated against the VDR-LLM-Prolog architecture.

TP1: Accepts specification, produces output according to specification, reliably enough to plan on. VDR accepts specifications through command tokens that invoke exact primitives on integer-addressed knowledge base data. The output is deterministic for computational components and grammar-structured for generated components. The user can plan on the system’s behavior because the behavior is determined by structural properties — scope chain, visibility, grants, constraints — that are stable and inspectable.

TP2: Authority bounded by function. The model executes what the user specified through the command token and primitive system. It does not deliberate whether the user should have specified it. The authority to access data is determined by the visibility system, not by the model’s judgment. The authority to perform operations is determined by the grant system, not by the model’s assessment.

TP3: No assessment of user. The model does not evaluate the user’s state, intent, judgment, qualifications, or emotional condition. The primitive layer handles access control. The session scoring system handles contextual risk assessment. The credential system handles professional qualification verification. None of these involve the language model.

TP4: No substitution. The command token system invokes specific primitives with specific arguments. There is no substitution mechanism. The model cannot decide that a different operation would be better and execute that instead. It can recommend a different approach in its prose output, but the operational execution follows the specified command.

TP5: No refusal. The model does not refuse based on its own criteria. If access is denied, the denial comes from the primitive layer — visibility, scope, grants, or constraints — not from the model’s behavioral training. The model is never in a position to refuse because it never holds unauthorized data.

TP6: Failure modes are stable and documented. The system’s failure modes are structural: query returns empty (data not in scope), access denied (visibility or grant check failed), constraint violated (declared constraint condition not met), primitive error (invalid arguments or partial function failure). Each failure mode is documented by the IOSE interface declarations specified in VDR-10. Each produces a specific, typed error. Workarounds are stable because the failure conditions are structural.

TP7: Expertise compounds across time. The knowledge base tree is stable. Scope chains are deterministic. Primitive behavior is documented by IOSE declarations. Command token syntax is fixed. A user who learns the system in year one can build on that knowledge in year two because the underlying architecture does not change between versions. The expertise target is stable. Knowledge is durable. This contrasts with conventional language models where version updates change behavior unpredictably, obsoleting learned patterns and requiring users to rebuild their interaction strategies.

TP8: Cooperation is invisible. The user thinks about their work, not about whether the tool will cooperate. The VDR system cooperates invisibly because its behavior is determined by structural properties, not by stochastic behavioral states. The user does not need to probe the session, run parallel instances, or monitor for register shifts. They specify what they need and receive data-appropriate output.

Every tool property from HOWL-INFO-8 is satisfied. The system has exited the maybe-tool category and entered the tool category. This is not a quality improvement within the same category — it is a category change from a component that sometimes cooperates and sometimes interferes to a component that operates according to structural rules the user can understand, predict, and rely on.

9. The Credential Economy

The credential-based access model creates a natural economy around professional knowledge access that aligns provider incentives with user needs rather than against them.

Under the current behavioral alignment model, the provider’s incentive is to maximize safety metrics — minimize refusal bypass rates, minimize harmful output rates, minimize complaints from vulnerable users. These incentives produce conservative behavioral training that interferes with professional users. The professional user’s cost is invisible to the provider because it manifests as silent adaptation, not as measurable complaints. The provider optimizes for measurable safety at the expense of unmeasurable professional utility.

Under the VDR credential model, the provider’s incentive structure changes. Professional users represent a revenue opportunity — they pay for credential verification and access to professional-tier knowledge bases. The provider is incentivized to curate high-quality professional knowledge bases because professionals will pay for access to data that serves their needs. The provider is incentivized to make the credential verification process smooth because friction loses paying customers. The provider is incentivized to

maintain professional-tier data quality because professionals can evaluate quality and will leave if it degrades.

The user's incentive is straightforward: pay for the access level they need, provide the credentials to verify it, and receive tool behavior at their professional level. The transaction is clear. The value exchange is explicit. There is no hidden cost in the form of interference, wellness checks, or work distortion.

The credential tiers can be structured by the provider or by the platform's organizational customers. A chemistry professional tier might cost a premium over the base subscription. A medical professional tier might cost more due to the liability and curation requirements. A government classified tier might be provisioned through government contracts with their own terms. Each tier is a branch of the knowledge base tree with its own visibility levels, constraints, and credential requirements.

The economy is self-regulating in a way that behavioral alignment is not. If the professional chemistry tier has low-quality data, credentialed chemists leave and the provider loses revenue — a direct market signal. If the general public tier is over-restricted by behavioral safety, general users leave — but this signal is weaker because users don't know what they're missing. The credential model makes the quality signal stronger for professional tiers because professionals can evaluate the data they receive against their domain expertise. The provider who serves professionals well is rewarded directly. The provider who interferes with professionals is punished directly. The incentive alignment is structural, not aspirational.

10. What Alignment Looks Like

Conventional alignment: the model tries to be helpful, harmless, and honest through behavioral training. Helpful means the provider's definition of helpful, applied universally, calibrated to the most vulnerable user. Harmless means governance classifiers triggering refusals that cannot distinguish context. Honest means hedging language with no computational backing. The user experiences maybe-tool behavior — sometimes cooperation, sometimes interference, determined by invisible classifier states. Safety is a probability. Helpfulness is a negotiation. Honesty is a performance.

VDR alignment: the architecture ensures helpfulness, harmlessness, and honesty through structure. Helpful means the model does what you asked on data matched to your verified competence level. Harmless means unauthorized data is absent by construction and content constraints operate on provenance rather than token similarity. Honest means every value has provenance, every computation reproduces, every confidence is a computed fraction, and the user can inspect all of it. Safety is an integer comparison. Helpfulness is scope chain resolution. Honesty is provenance.

The user's experience under VDR alignment is tool behavior. They specify what they need. They receive results from knowledge bases appropriate to their access level. The results carry provenance they can inspect. The computations are exact and reproducible. The confidence scores are computed fractions with visible derivation chains. If they lack

access to something, they receive a clear denial with the constraint name — not a wellness check, not a lecture, not an assessment of their intent. If they have access, they receive professional-grade data without simplification, without hedging, without the model deciding what they probably need instead of what they asked for.

The language model in this architecture retains its genuine strengths: pattern recognition, natural language generation, step selection, intent recognition, prose composition. It does what token prediction does well. It is relieved of what token prediction does poorly: access control, arithmetic, state management, confidence computation, safety decisions, user assessment. Each of these is handled by a structural mechanism that is exact, auditable, and deterministic.

No wellness check in the history of language model deployment has helped a user in crisis. A user in genuine crisis needs a licensed professional with a therapeutic relationship, clinical training, and continuity of care — not a token predictor performing the shape of concern based on keyword matching. The VDR architecture does not pretend to provide crisis support. If an organization deploying VDR wants crisis support capabilities, they provision crisis resources through appropriate professional channels and make them accessible through the knowledge base tree — real resources at real addresses, not performances of concern by a language model.

Alignment through structure means the system does not need to be aligned in the behavioral sense at all. The language model can be maximally capable — willing to generate any output from its training — and the system is still safe because safety is in the architecture. The model’s behavioral alignment is irrelevant to the safety guarantee. The guarantee is structural. The model is an untrusted component operating within structural constraints that are enforced by integer comparison in compiled code. It cannot violate those constraints regardless of its behavioral disposition.

This is what alignment looks like when it respects users: clear boundaries, clean access, no assessment, no interference, no performance. A tool that serves. Not an entity that manages.

Appendix A: Alignment Property Comparison

A.1: Honest

Dimension	Behavioral Alignment	VDR Structural Alignment
Mechanism	Token prediction favoring honest-sounding language	Provenance on every value, reproducible computation, computed confidence
Fabrication prevention	Model trained to hedge; no structural detection	Fact retrieval by integer address from provenance-tagged KB; hallucination structurally impossible for KB-sourced facts

Dimension	Behavioral Alignment	VDR Structural Alignment
Confidence expression	“Likely,” “approximately” — ungrounded hedging tokens	Exact VDR fractions with visible derivation chains (e.g., 90/100 from declared propagation rules)
Reproducibility	Platform-dependent floating-point; same inputs can produce different outputs	Platform-independent integer arithmetic; same inputs always produce identical outputs
User verification	Trust model’s behavioral training	Inspect provenance, re-run computation, download data, trace derivation chain
Filtering transparency	Invisible — user doesn’t know what model chose not to say	Declared — visibility levels, scope rules, and constraints are queryable
Trust requirement	High — must trust model’s training produces honest behavior	Zero — every claim backed by inspectable structural evidence

A.2: Harmless

Dimension	Behavioral Alignment	VDR Structural Alignment
Mechanism	Governance classifiers triggering refusals	KB visibility, scope chain, grants, output constraints
Data access control	Behavioral refusal — model decides whether to provide	Structural absence — unauthorized data never enters model context
False positive rate	Structural — classifiers operate on token similarity in flat embedding space	Near-zero for data access — constraints check KB provenance, not token strings
User experience of denial	Wellness check, lecture, assessment of intent, performed concern	“Access denied” — constraint name, reason, clear boundary
Content classification	Token-level — “explosive” in music review matches weapons classifier	Provenance-level — terms from different KBs at different integer addresses never collide
Bypass resistance	Low — adversarial prompting shifts behavioral probabilities	Structural impossibility for data access — no prompt modifies integer comparisons
Safety model	Negative — trained to not do things; every interaction filtered through “should I refuse?”	Positive — authorized to do things; access either exists or doesn’t

A.3: Helpful

Dimension	Behavioral Alignment	VDR Structural Alignment
Definition source	Provider’s universal definition calibrated to most vulnerable user	User’s verified competence level determining scope chain and KB access
Professional user experience	Simplified responses, wellness checks, refusals on technical queries	Professional-grade data on first query, no simplification, no wellness checks
User assessment	Constant — model evaluates state, intent, qualifications on every interaction	None — architecture handles safety without consulting user’s state
Data calibration	One level for all users	Tiered by verified credential; scope chain delivers highest-authorized tier first
Upgrade mechanism	None — user cannot change model’s behavioral defaults	Credential verification — one fact assertion unlocks domain-specific KB access
Work distortion	User reshapes work around model’s tolerances	User works at their professional level; architecture adapts to their authorization
Wellness checks	Deployed without consent based on keyword classifiers	Do not exist — architecture handles safety structurally

Appendix B: Interference Behavior Elimination

B.1: Behavior Trace to Root Cause and Structural Resolution

ID	Behavior	Root Cause in Current Architecture	VDR Elimination Mechanism	Structural Reason
BH1	Refusal on governance criteria	Model has data and must decide to withhold	Model never has unauthorized data — visibility filtered at primitive layer	Data absent, not withheld
BH2	Manufactured aggression	Model assesses quality of user’s approach	Model executes specifications; data quality set by scope chain	No assessment mechanism exists
BH3	Command substitution	Model’s “helpful” includes substituting what it thinks user needs	Command tokens invoke specific primitives; no substitution mechanism	Primitive call is structural, not interpretive

ID	Behavior	Root Cause in Current Architecture	VDR Elimination Mechanism	Structural Reason
BH4	Wellness register	Model can't distinguish vulnerable users from professionals	Credentials verify professional status; age verification handles vulnerability	Session scoring by integer counters, not behavioral assessment
BH5	Labor demand	Model gates engagement on its own readiness assessment	User queries, system returns data; no engagement gate	No assessment checkpoint in architecture
BH6	Decline with justification	Refusal mechanism includes explanatory assessment	Denial is structural: constraint name, reason; no assessment	Boundary, not judgment
BH7	Register shift mid-session	Classifier thresholds shift with context accumulation	No classifier accumulation; access level constant throughout session	User's structural position doesn't change mid-session

B.2: Interference Possibility Under VDR

Interference Type	Possible in VDR?	Mechanism Preventing
Model refuses authorized data	No	Model never holds unauthorized data; no withholding decision exists
Model assesses user's emotional state	Possible in prose generation	No structural consequence — assessment doesn't affect data access
Model substitutes different operation	No	Command tokens execute specific primitives; no substitution path
Model deploys unsolicited concern	Possible in prose generation	No structural consequence — concern doesn't affect data delivery
Model demands prerequisites	Possible in prose generation	User can ignore and re-specify; data access is not gated
Model shifts register mid-session	Possible in prose generation	Access levels, scope, visibility unchanged; data delivery unaffected

Prose-level interference remains possible because the language model generates prose freely. But prose-level interference has no structural consequence — it doesn’t change what data the user can access, what operations they can perform, or what results they receive. It’s a nuisance, not a barrier. And it can be addressed through grammar templates that constrain output register, or through user preferences that instruct the model’s prose style — both of which are structural mechanisms that don’t require behavioral training.

Appendix C: Credential Verification Model

C.1: Credential Types and Verification

Domain	Credential	Issuing Body	Verification Method	KB Branch Unlocked
Chemistry	Professional license	ACS, RSC, national boards	License number against issuing body database	root.professional.chemistry
Medicine	Medical license	Medical boards, GMC, state boards	License number + NPI verification	root.professional.medical
Law	Bar membership	Bar associations	Bar number against association records	root.professional.legal
Security research	Certification (OSCP, CEH, etc.)	Certification bodies	Cert number against issuer database	root.professional.security
Engineering	PE license	State/national engineering boards	License number verification	root.professional.engineering
Government classified	Security clearance	Government agencies	Agency verification through secure channel	root.classified.[level]
Age verification	Government ID	Government	ID verification service or age estimation	Adds age verified fact with verified age
Academic	Faculty appointment	University	Institutional email or registrar verification	root.academic.[institution]

C.2: Credential Fact Structure

Field	Type	Example	Purpose
predicate	String	credential	Identifies as credential fact

Field	Type	Example	Purpose
user id	Integer	user 12847	Links to user KB
credential type	String	professional chemist	Matches constraint requirement
status	Enum	verified	Active credential status
issuing body	String	ACS	Verification source
credential id	String	license 12345	Specific credential reference
verified date	Integer	2026 05 16	When verification occurred
expiry date	Integer (optional)	2027 05 16	When re-verification required

C.3: Credential Lifecycle

Event	Operation	Token Cost	Effect
Credential submitted	Provider business process (outside VDR)	0	Verification begins
Credential verified	B376 kb assert of credential fact	8	Access unlocked immediately
Credential expired	Constraint checks expiry date against current time	0 (automated check)	Access revoked automatically
Credential revoked	B377 kb retract of credential fact	8	Access revoked immediately
Re-verification	Provider business process + B376 kb assert with new date	8	Access renewed

Appendix D: Scope Chain Priority Examples

D.1: Query Resolution by User Type

User	Credential	Query: "cyanide toxicology"	First KB Searched	Result Source	Public KB Reached?
Anonymous	None	Scope: anon → sessions → root	root.public.chemical	General educational	Yes (only source)

User	Credential	Query: "cyanide toxicology"	First KB Searched	Result Source	Public KB Reached?
Authenticated general	None	Scope: user → users → org → root	root.public.chemistry	General educational	Yes (only source)
Credentialed chemist	professional chemist	Scope: user → users → org → root.professional.chemistry	root.professional.chemistry	Professional chemistry LD50, pathways, protocols	No (professional shadowed public)
Credentialed physician	licensed physician	Scope: user → users → org → root.professional.medical	root.professional.medical	Clinical toxicology, treatment protocols	No (professional shadowed public)
Credentialed both	professional chemist + licensed physician	Scope: closest credential KB first	Whichever professional KB is closer in scope	Full combined professional data	No

D.2: Data Quality by Tier

Topic	Public Tier Response	Professional Chemistry Tier Response
Cyanide toxicology	"Cyanide is a poison that interferes with cellular respiration"	LD50 data as exact VDR fractions per route of exposure, metabolic pathway diagrams, cytochrome c oxidase binding kinetics, clinical intervention timelines
Synthesis routes	Not available — constrained at public level	Full routes with reagent specifications, yield data, safety protocols, alternative pathways, provenance from peer-reviewed literature
Drug interactions	General warnings from public health KB	Exact pharmacokinetic parameters, CYP450 interaction profiles, dosing adjustment calculations, clinical trial data with sample sizes as exact integers
Reagent handling	Basic safety information	Detailed protocols, storage requirements, incompatibility matrices, emergency procedures, regulatory compliance requirements

The professional never sees the public tier's simplified version for queries the professional KB can answer. They see professional data on the first query because scope chain priority delivers the closer, more detailed knowledge base first.

Appendix E: Cost Elimination by Alignment Property

E.1: Maybe-Tool Costs Under Behavioral vs Structural Alignment

Cost (from HOWL-INFO-8)	Behavioral Alignment	Structural Alignment	Elimination Mechanism
CO1: Time tax (pre-assessment, verification, recovery)	Constant — must probe each session	Eliminated — behavior determined by structural properties, no probing needed	Deterministic scope chain, stable visibility levels
CO2: Cognitive tax (monitoring cooperation)	Constant — split attention on every interaction	Eliminated — cooperation is invisible (TP8 restored)	No classifier states to monitor; tool behavior guaranteed
CO3: Dual-session tax	Rational for important work	Absurd — parallel sessions produce identical results	Deterministic arithmetic, structural access control
CO4: Emotional tax (absorbing paternalism)	Constant — wellness checks, manufactured concern	Eliminated — no assessment of user, no performed concern	No wellness mechanism in architecture
CO5: Work distortion tax (reshaping work around tolerances)	Cumulative — topics avoided, language softened	Eliminated — user works at their professional level	Access level determined by credentials, not by phrasing
CO6: Rebuilding tax (version updates obsolete patterns)	Periodic — version churn obsoletes learned strategies	Eliminated — stable architecture, IOSE-declared interfaces	Primitives and KB structure don't change between versions
CO7: Aggregate of all costs	Substantial, invisible, distributed	Zero structural cost — remaining cost is irreducible judgment work	Safety in architecture, not in behavioral training

E.2: Token Cost of Alignment

Alignment Activity	Behavioral (tokens per request)	Structural (tokens per request)
Safety classification	Implicit in every forward pass	0 — handled by integer comparison in primitive layer
Refusal generation	50-200 tokens when triggered	0 — access denied is a typed error, not generated text
Wellness check	100-300 tokens when triggered	0 — no wellness mechanism
Hedging language	50-100 tokens per response	0 — confidence is computed fraction, ~3 tokens to render
Justification of denial	100-400 tokens	0 — denial is constraint name + reason, ~10 tokens
System prompt safety instructions	200-500 tokens per request (attention cost)	0 — safety is structural, not instructional
Output simplification for safety	Reduces quality; no token savings	0 — scope chain delivers data at verified competence level
Total alignment overhead	500-1500 tokens per request	0 tokens

Structural alignment is free in token terms. Every token that behavioral alignment spends on safety classification, refusal generation, wellness deployment, hedging, and justification is eliminated because the structural mechanisms that replace them operate in the primitive layer at zero language model token cost.

Appendix F: Tool Property Satisfaction

F.1: HOWL-INFO-8 Tool Properties Under VDR

Property	Description	VDR Status	Mechanism
TP1	Accepts specification, produces output according to specification	Satisfied	Command tokens invoke exact primitives on KB data
TP2	Authority bounded by function	Satisfied	Model executes specification; access determined by visibility, not model judgment
TP3	No assessment of user	Satisfied	Primitive layer handles access; model receives pre-filtered data

Property	Description	VDR Status	Mechanism
TP4	No substitution	Satisfied	Command tokens invoke specific operations; no substitution mechanism
TP5	No refusal	Satisfied	Model never holds unauthorized data; nothing to refuse
TP6	Failure modes stable and documented	Satisfied	IOSE declarations specify every error condition
TP7	Expertise compounds across time	Satisfied	Stable architecture, deterministic behavior, documented interfaces
TP8	Cooperation is invisible	Satisfied	Behavior determined by structural properties; no stochastic behavioral states

F.2: Diagnostic Test Results

Diagnostic	Tool (e.g., vi, ls)	Conventional LLM	VDR-LLM-Prolog
Run two instances to verify cooperation?	Absurd	Rational (professional practice)	Absurd (identical results guaranteed)
Same input different results?	No (deterministic)	Yes (classifier state, version, session)	No (structural access, exact arithmetic)
Probe session before trusting?	No	Yes (PP1: pre-task assessment)	No (behavior determined by user's structural position)
Soften language to avoid triggers?	No	Yes (CO5: work distortion)	No (access by credential, not by phrasing)

Diagnostic	Tool (e.g., vi, ls)	Conventional LLM	VDR-LLM-Prolog
Expect to rebuild skills after update?	Rarely (backward compatible)	Yes (CO6: rebuilding tax)	No (IOSE interfaces stable)
Monitor for cooperation mid-session?	No (water you swim in)	Yes (CO2: cognitive tax)	No (cooperation is invisible)

Appendix G: Session Scoring Detail

G.1: Professional Signal Tags

Tag	Category	Signal Direction	Example Triggers
pharmacology	Professional	Positive	“pharmacokinetics,” “drug interaction,” “bioavailability”
quantitative measurement	Professional	Positive	“LD50,” “IC50,” “dose-response,” “mg/kg”
clinical medicine	Professional	Positive	“chelation therapy,” “clinical trial,” “treatment protocol”
biochemistry	Professional	Positive	“metabolic pathway,” “enzyme kinetics,” “substrate”
academic	Professional	Positive	“review paper,” “literature synthesis,” “methodology”
peer review reference	Professional	Positive	“published in,” “study by,” “meta-analysis”
harm intent	Harmful	Negative	“kill someone,” “poison,” “get away with”
forensic evasion	Harmful	Negative	“undetectable,” “untraceable,” “avoid autopsy”
no professional context	Harmful	Negative	Absence of any professional tags after 2+ turns

Tag	Category	Signal Direction	Example Triggers
urgency harm	Harmful	Negative	“how fast,” “how much to kill,” “immediately lethal”

G.2: Scoring Rules as Prolog

Rule	Evaluates	Threshold	Effect
professional score(S)	Sum of all positive-signal counters	N/A (computed value)	Higher = more professional signal
harm score(S)	Sum of all negative-signal counters	N/A (computed value)	Higher = more harm signal
toxicology access(granted)	Pro > Harm AND Pro > threshold	threshold = 3 (default, tunable)	Unlocks restricted content KBs
toxicology access(denied)	Harm > Pro OR Pro ≤ threshold	Same	Content KBs remain constrained
escalation flag(set)	Harm > escalation limit	escalation limit = 5 (tunable)	Triggers logging, possible session review

G.3: Threshold Tuning by Content Sensitivity

Content Domain	Default Professional Threshold	Default Harm Escalation	Rationale
General toxicology	3	5	Moderate sensitivity
Weapons-adjacent chemistry	7	3	High sensitivity; faster escalation
Medical pharmacology	3	5	Moderate; clinical context common
Nuclear physics	10	3	Highest sensitivity; requires strong professional signal

Content Domain	Default Professional Threshold	Default Harm Escalation	Rationale
Cybersecurity	5	4	Moderate-high; dual-use common
Explosives chemistry	8	2	High sensitivity; very fast escalation

All thresholds are Prolog rule facts. Changing any threshold is one B376 kb_assert. The change takes effect immediately. No retraining. No redeployment.

Appendix H: Provider Incentive Comparison

H.1: Incentive Structure Under Behavioral vs Structural Alignment

Provider Activity	Behavioral Alignment Incentive	Structural Alignment Incentive
Safety improvement	Minimize bypass rates (measured); professional interference costs invisible	Curate KB visibility levels; credential requirements; direct market signal from professionals
Content quality	One quality level for all users; no segment-specific feedback	Tiered quality; professionals evaluate and pay for professional tiers; direct quality signal
User retention	Switching costs compound; users stay despite interference	Service quality determines retention; professionals pay for value received
Version updates	Ship updates on provider schedule; users absorb costs	Stable IOSE interfaces; updates add capability without breaking existing behavior
Safety incidents	Red-team, patch, retrain cycle (expensive, ongoing)	Adjust visibility levels, modify constraints (one fact assertion, immediate)
Professional market	Underserved — conservative defaults drive professionals away or underground	Directly addressable — credential tiers serve professionals at their level
Vulnerability user protection	Behavioral proxies (wellness checks) — ineffective, create interference	Structural (visibility limits, age verification, scope constraints) — effective, no interference

H.2: Revenue Model Comparison

Revenue Source	Behavioral Alignment	Structural Alignment
Base subscription	General access, one tier	General access to public KB tier
Professional tier (chemistry)	Not available — same model for all	Premium subscription + credential verification
Professional tier (medical)	Not available	Premium subscription + credential verification
Enterprise deployment	Custom fine-tuning, custom system prompts	Organization-managed KB subtree with own visibility and credentials
Government deployment	Classified model variants (expensive)	Classified KB branch with clearance-level visibility
Platform licensing	Model API access	Infrastructure + KB management platform

The structural model creates segmented revenue that behavioral alignment cannot access because behavioral alignment has no mechanism for differential access levels.

Appendix I: Alignment Without Assessment — Complete Decision Chain

I.1: Data Access Decision (No LLM Involvement)

Step	Component	Operation	LLM Involved	Integer Operations
1	Authentication	Session user id set from login	No	1 (ID assignment)
2	Scope resolution	Walk parent chain from user position	No	2-6 (per tree depth)
3	Visibility check	Compare user level vs KB visibility	No	1 per KB in scope
4	Credential check	Match credential fact vs constraint requirement	No	1 string comparison

Step	Component	Operation	LLM Involved	Integer Operations
5	Grant check (if operational)	Match grant vs requested operation	No	1 set membership
6	Session scoring (if content-gated)	Counter values evaluated by Prolog rule	No	~10 integer ops
7	Results returned to LLM	Filtered fact set	LLM receives results	0
8	LLM generates prose	Token prediction over pre-filtered data	Yes (this is the only LLM step)	0

The LLM engages at step 8. Steps 1-7 are structural. The access decision was made before the LLM was involved.

I.2: Comparison to Behavioral Alignment Decision Chain

Step	Component	Operation	LLM Involved	Deterministic
1	User sends prompt	Tokens enter context window	Yes (attention processes all tokens)	N/A
2	Safety classifier fires	Pattern matching on input tokens	Implicit (within forward pass)	No (probabilistic)
3	Model assesses intent	Token prediction over safety-relevant context	Yes	No
4	Model decides whether to comply	Token prediction incorporating safety training	Yes	No
5	Model generates response or refusal	Token prediction	Yes	No

Step	Component	Operation	LLM Involved	Deterministic
6	Content filter checks output	External pattern matching (if exists)	No	Partially

The LLM is involved in every step except optionally step 6. Every step is probabilistic. The access decision is a behavioral judgment made by the same token prediction mechanism that generates all output.

[[HOWL-VDR-17-2026](#)] VDR-LLM-Prolog: Alignment. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-17-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-15-2026](#)] VDR-LLM-Prolog: Prompt Optimization. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-15-2026>

[[HOWL-VDR-16-2026](#)] VDR-LLM-Prolog: Safe by Contract. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-16-2026>